



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Parallelism V, Virtual Memory I



MIMD: Design Questions

1. How do different processors (cores) share data?
 - Via **shared-memory multiprocessor** (SMP)
2. How do different processors coordinate/communicate?
 - **Shared variables in memory** and load/store instructions
 - Coordinated access to shared data through **synchronization primitives** (e.g., locks) that restrict access to one processor at a time



Cache Coherence

- MIMD: implementation —
 - Cache Coherence
 - Synchronization with Locks
- Virtual Memory —
 - Virtual Memory and Virtual Addresses
 - Paged Memory

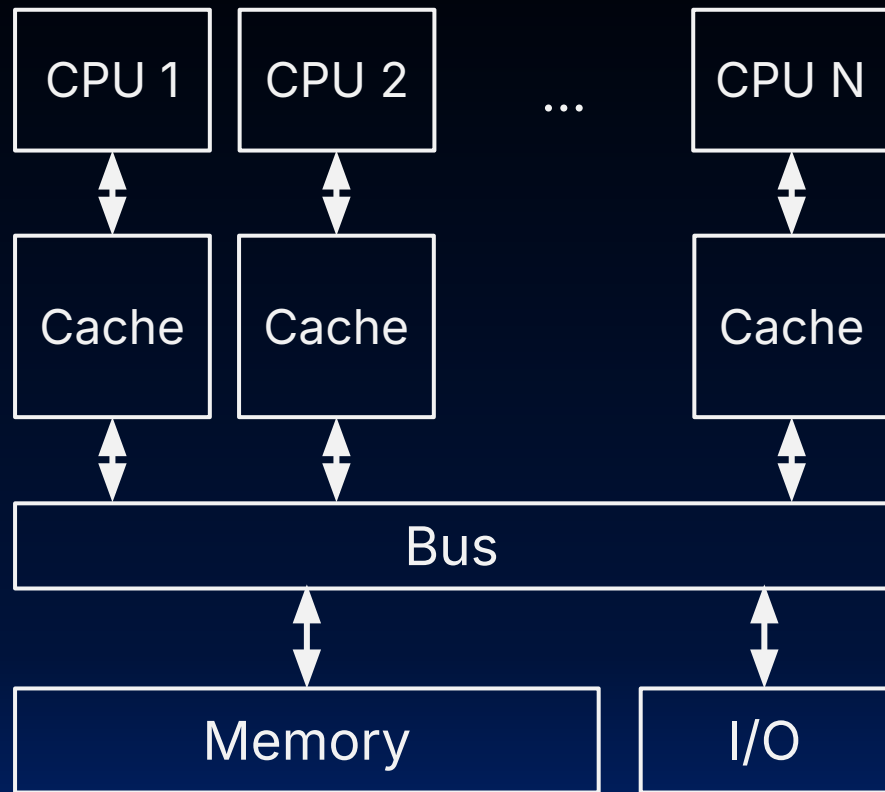
Multiprocessor Caches

- A **shared-memory multiprocessor** (SMP)* offers multiple cores / CPUs a single, shared, coherent memory.

?

All CPUs communicate with each other and memory through a bus.

One bank of memory is shared by all CPUs.

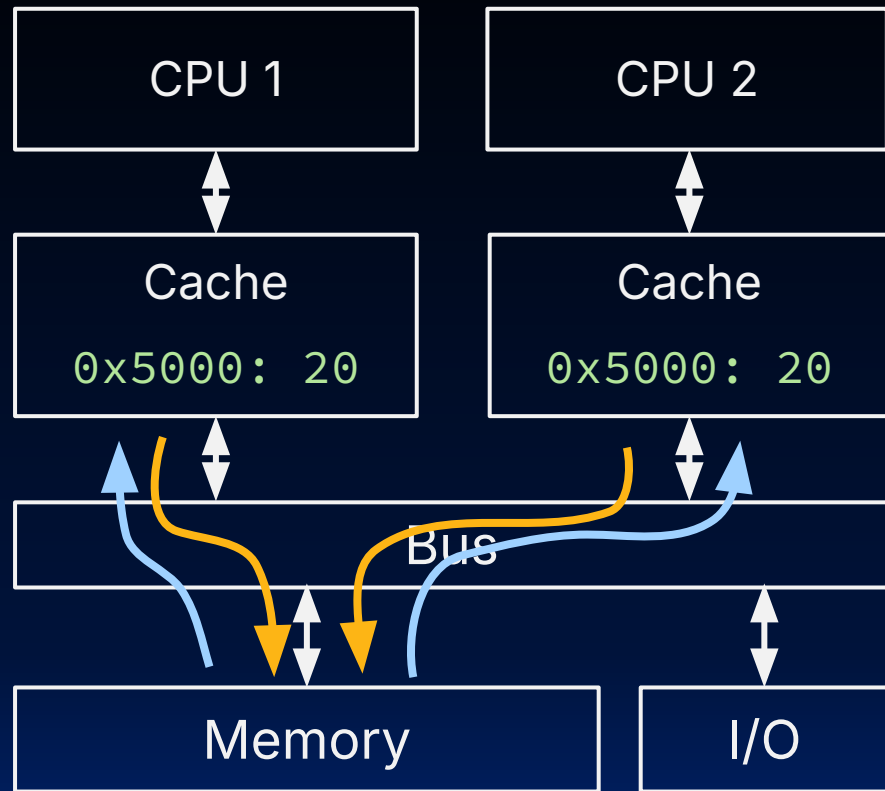


Cache Coherence Problem (1/3)

Assume value "20" initially @ Mem[0x5000]:

1. CPU 1 reads Mem[0x5000]
2. CPU 2 reads Mem[0x5000]

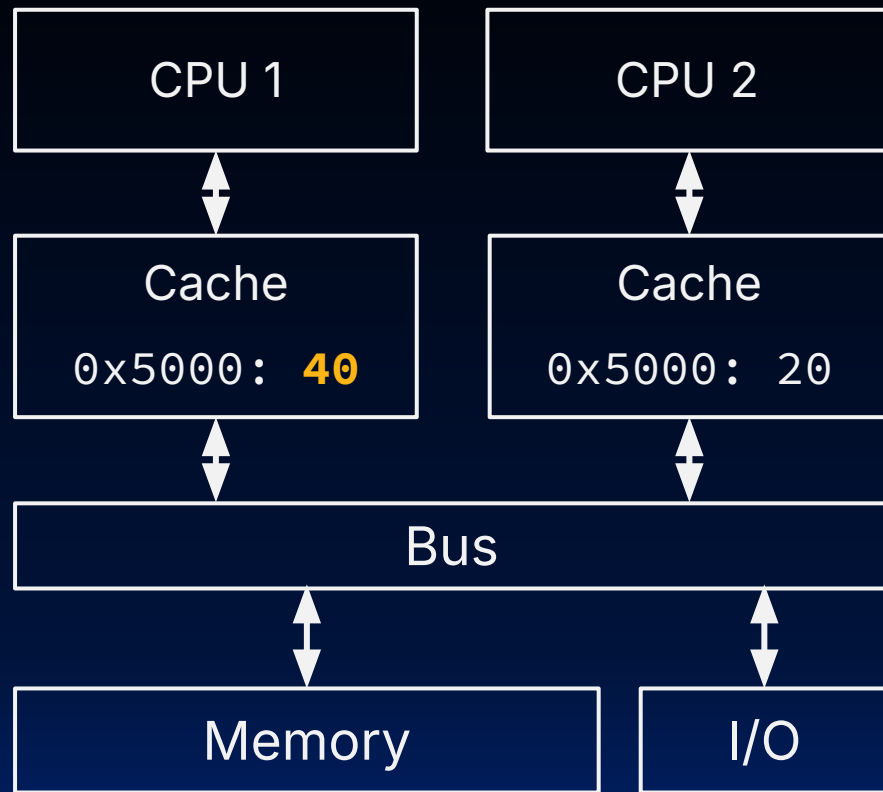
request
response



Cache Coherence Problem (2/3)

Assume value "20" initially @
Mem[0x5000]:

1. CPU 1 reads
Mem[0x5000]
2. CPU 2 reads
Mem[0x5000]
3. CPU 1 writes 40 to
Mem[0x5000]

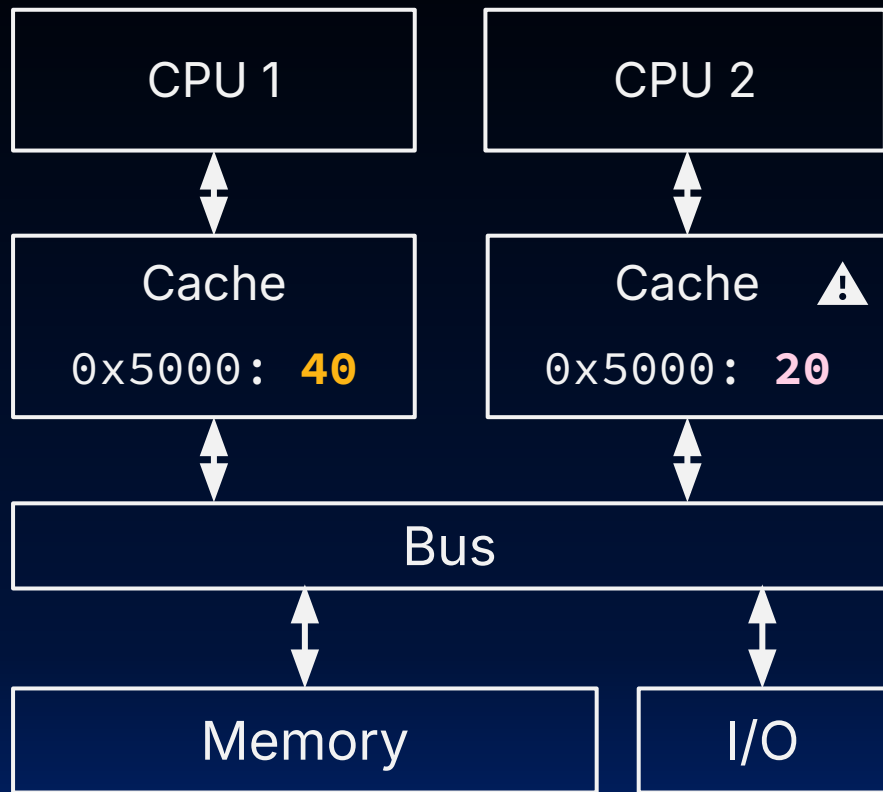




Cache Coherence Problem (2/3): What happened?

- When CPU 1 performed a write, CPU 1's cache is **up-to-date**, but CPU 2's cache is now stale, and it **doesn't know**.

New cache miss type: **coherence miss** (i.e., communication miss), caused by writes to shared data made by other processors.

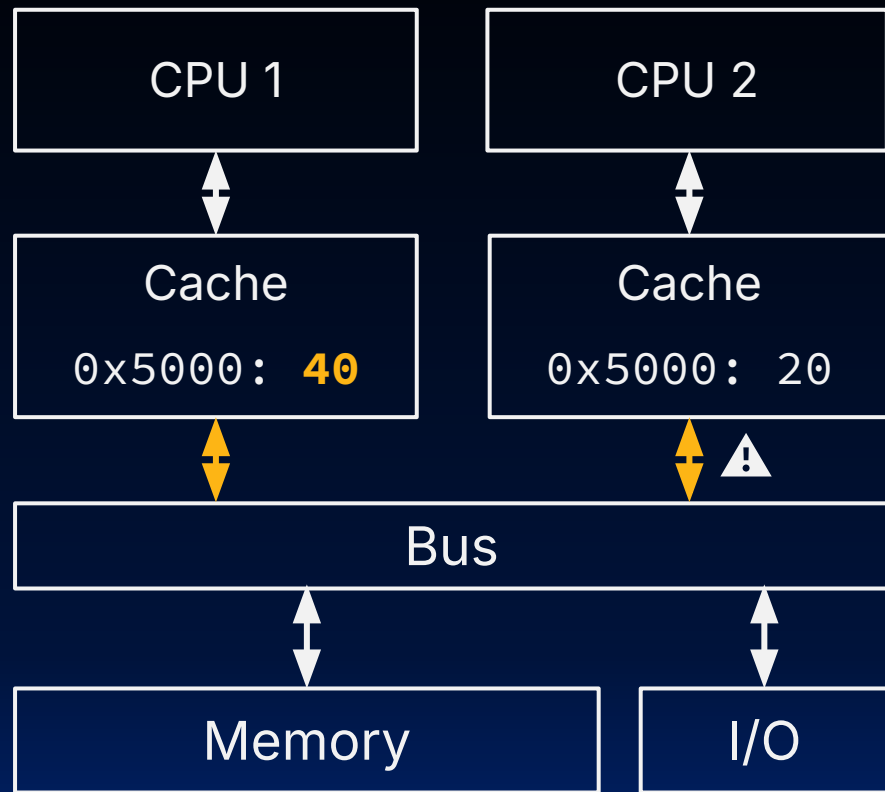


Cache Coherence Solution: Snoop for writes

[out of scope]

[out of scope. Take CS 152]

- Each cache controller "snoops" for write transactions on the bus
- When CPU 1 writes, it **invalidates** other copies in other CPUs' caches.





Synchronization with Locks

How might OpenMP synchronization work, under the surface?

- MIMD: implementation —
 - Cache Coherence
 - Synchronization with Locks
- Virtual Memory —
 - Virtual Memory and Virtual Addresses
 - Paged Memory

No data race with critical section

```
1  #include <stdio.h>
2  #include <omp.h>
3  int main() {
4      int x = 0;
5      #pragma omp parallel
6      {
7          #pragma omp critical
8          {
9              x += 1;
10         }
11     }
12 }
13
14
15
16
17
18
19
```



How to implement?

Analogy: Buying Milk, Two Approaches

- You and your roommate want to agree on how to buy milk.
 - Both of you will use the same procedure. Assume shared fridge.

// procedure 1

```
if milk not in fridge:  
    buy milk at store  
    put milk in fridge
```

// procedure 2

```
if note not on fridge:  
    if milk not in fridge:  
        put note on fridge  
        buy milk at store  
        put milk in fridge  
        take note off fridge
```

For each of these algorithms, will you avoid a data race?



Analogy: Buying Milk, Procedure 2

- Even with a shared note, we can run into a two-milk situation:

```
// you
if note not on fridge:
    if milk not in fridge:

        put note on fridge

        buy milk at store
        put milk in fridge
        take note off fridge
```

```
// roommate
                                     ↓ time
if note not on fridge:
    if milk not in fridge:

        put note on fridge

        buy milk at store
        put milk in fridge
        take note off fridge
```

Even when threads execute in parallel, they still sequentially access shared resources.

Synchronization with (Spin) Locks

- Solution: Use a **lock** to grant one thread exclusive access to a resource or critical section.
 - Analogy: bathroom stall locks.
One person at a time;
others wait.
- Two operations, formally:
 - **Acquire**: Try to acquire the lock.
If successful, keep going.
Otherwise, wait a bit and try acquiring again later.
 - **Release**: Unlock the lock and continue.

```
// procedure 3, with lock
acquire fridgelock
  if milk not in fridge:
    buy milk at store
    put milk in fridge
  release fridgelock
```



Analogy: Buying Milk, Procedure 3 (Spinlocks)

- When your roommate does not have the lock, they **wait**.

time
↓

```
// you  
acquire fridgelock
```

```
    if milk not in fridge:  
        buy milk at store  
        put milk in fridge  
    release fridgelock
```

```
// roommate
```

```
acquire fridgelock  <wait>
```

```
    if milk not in fridge:  
        buy milk at store  
        put milk in fridge  
    release fridgelock
```

Deadlock: An Even Worse Situation

- **Deadlock:** A system state in which no progress is possible because everything is locked waiting for something else.



[reddit](#)

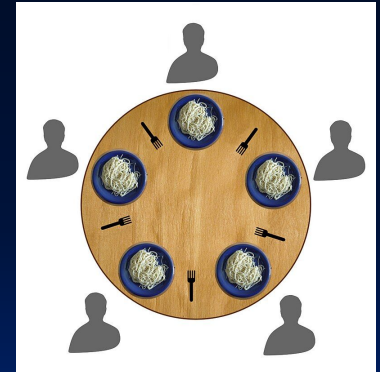
Deadlock situations are common when working directly with locks. Be careful!

Dining Philosophers Problem

- Dining Philosophers problem (Dijkstra, 1965)
 - Special spaghetti must be eaten with two forks (**one left fork, one right fork**)
 - Each philosopher can only think OR eat
- Simple algorithm: Grab left fork, grab right fork, then eat
- Can lead to **deadlock**:
 - All philosophers grab **left fork** at same time
 - All are waiting for **right fork**
 - All stuck thinking forever



(starving to death,
in minecraft)



Yan, SP26

[wikipedia](https://en.wikipedia.org/wiki/Dining_philosophers_problem)



OpenMP: Locks?

- OpenMP also has locks, but we suggest just using the existing **critical section** directives.
 - Again, locks are a **means** of implementing synchronization
- For more information:
<https://www.openmp.org/spec-html/5.0/openmpse31.html>

Locks (and other synchronization techniques) inherently enforce *some* serialization of threads.

- Amdahl's Law strikes again!
- Take CS 162!

SP26



Assembly: Atomic Instructions?

[reference]

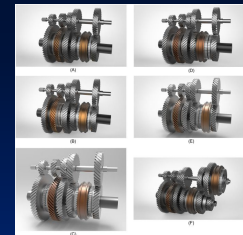
- **Atomic instruction**: all other instructions must appear to happen strictly before or after this one
 - Dining philosophers problem: atomically pick up both forks or none

For CS 61C: Do not use atomic instructions directly! Used to implement synchronization primitives.

- Two approaches to implementing **spin-locks**:
 - **single atomic instruction**: single instruction swaps register with memory (e.g., amoswap or compare-and-swap) [Zaamo extension]
 - **pair of instructions** (e.g., load-reserved, store-conditional) are effectively atomic [Zalsrc extension]
 - [out of scope] See lecture videos



(change of
gears)





CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Virtual Memory I



OS, VERY briefly

Take CS 162!!!

1. The **first thing** that runs when the computer starts
 - Starts services (100+).
 - File system, Network stack (Ethernet, WiFi, Bluetooth, ...), TTY (keyboard), etc.
2. **I/O** with rest of computer
 - Provides interaction with the outside world:
 - Finds and controls all devices in the machine in a general way:
 - Relies on hardware specific "device drivers"
3. Loads, runs and manages programs:
 - **Isolation**: Each program runs (i.e., appears to run) in its own little world.
 - **Resource-sharing**: Multiple programs share the same resources:
 - Memory
 - I/O devices: disk, keyboard, display, network, etc.
 - **Time-sharing**: Processor (CPU) runs multiple processes.

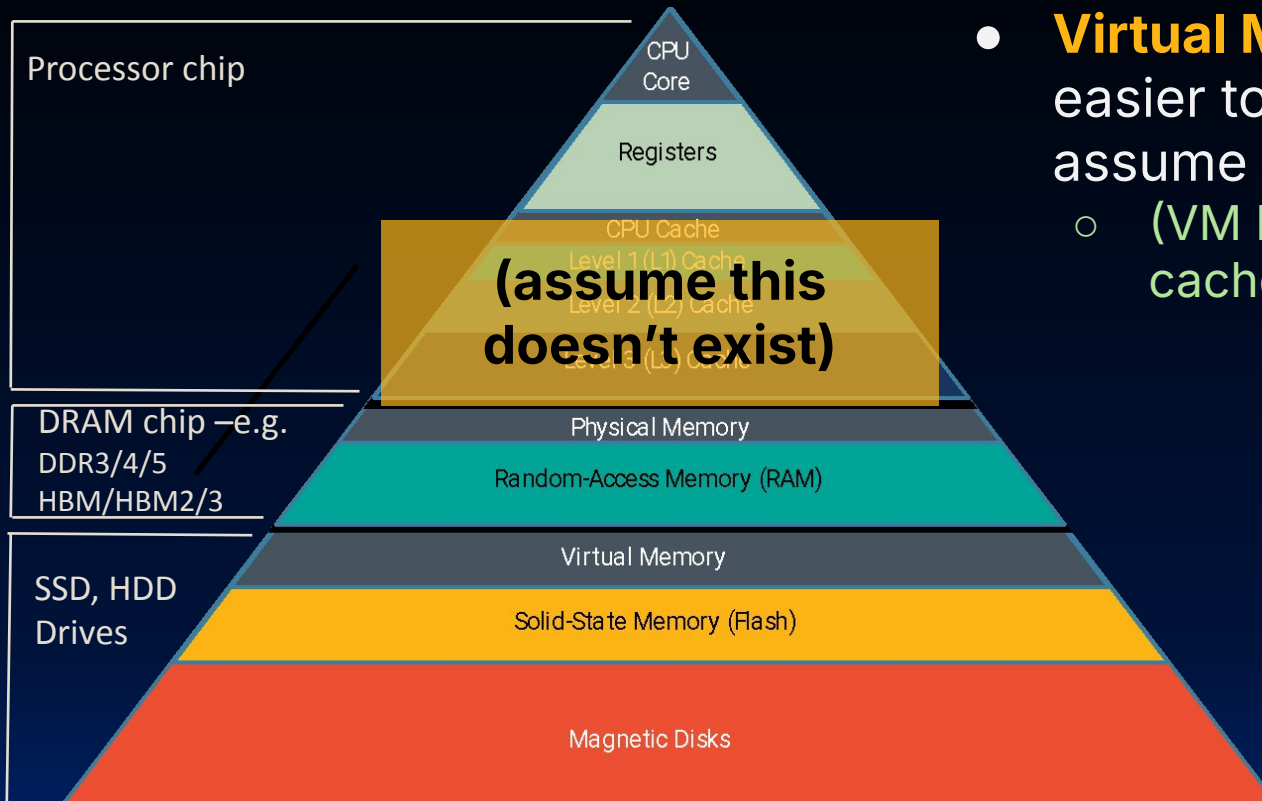


OS: Context Switches

- **Process**: running program/application.
- **Thread**: thread within a program.
- The OS manages multiprogramming and multithreading via **context switches**:
 - Interrupt execution of the current instruction
 - Save the current state of the process or thread (registers, PC, etc.)
 - Load in state of another process or thread (registers, PC, etc.)
 - Execute the next instruction

What about other layers of the memory hierarchy?

⚠ Assume Caches Don't Exist For Now ⚠



- **Virtual Memory** is much easier to understand if we assume **no caches**.
 - (VM historically predated caches, anyway)



Virtual Memory and Virtual Addresses

- MIMD: implementation —
 - Cache Coherence
 - Synchronization with Locks
- Virtual Memory —
 - Virtual Memory and Virtual Addresses
 - Paged Memory

The Case for Virtual Memory (1/2)

1. What if main memory is smaller than the program address space?

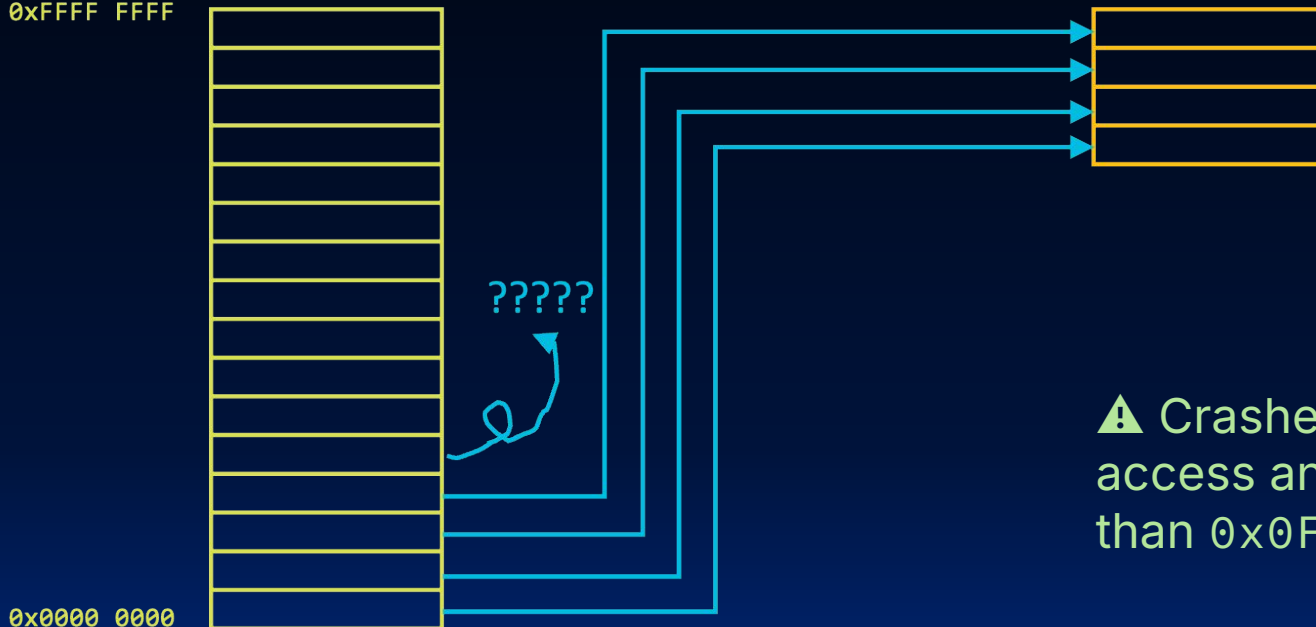
RV32I provides a 32-bit address space.

→ 2^{32} B = 4 GiB addressable memory

0xFFFF FFFF

Suppose **RAM** is 1GiB.

→ 2^{30} B addressable memory.



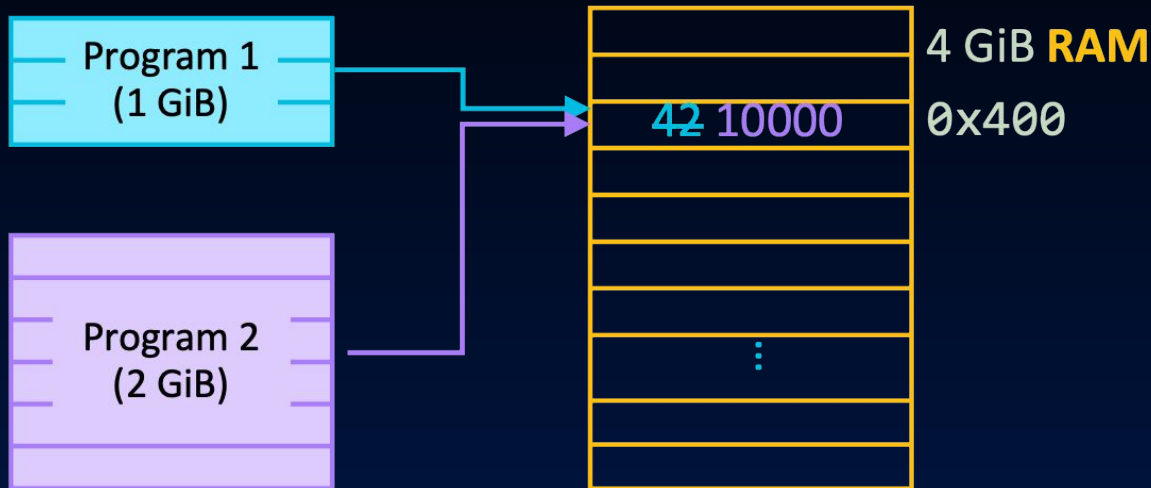
⚠ Crashes if we try to access an address higher than 0x0FFF FFFF!

The Case for Virtual Memory (2/2)

1. What if main memory is smaller than the program address space?
2. What if two programs access the same memory address?

Program 1 stores your bank account balance at address `0x400`

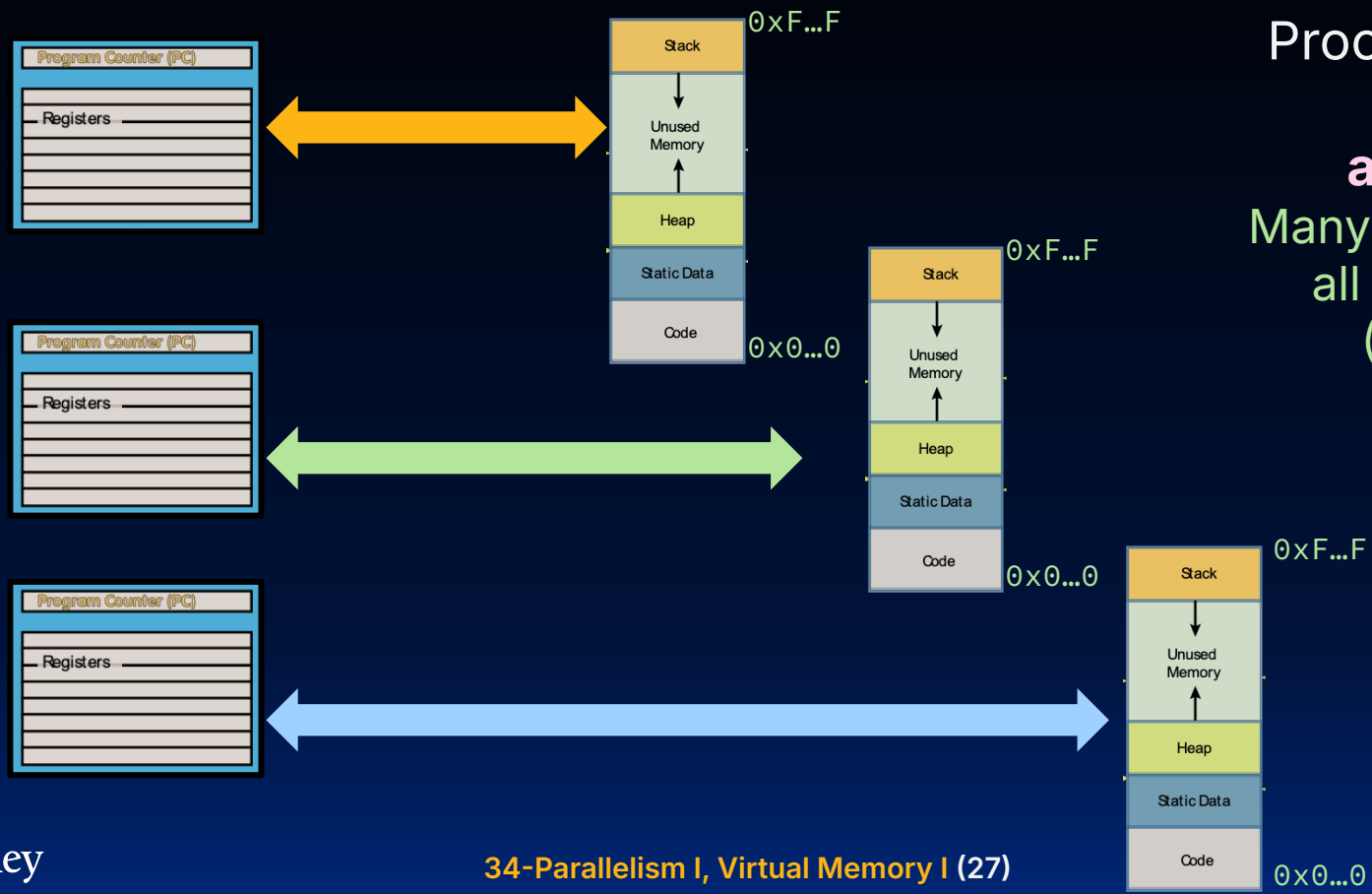
Program 2 stores your video game score at address `0x400`



- ⚠ If all processes can access any 32-bit memory address, they can corrupt/crash others.
- Need **protection** (i.e., **isolation**) between processes.

The Illusion of Virtual Address Space (1/2)

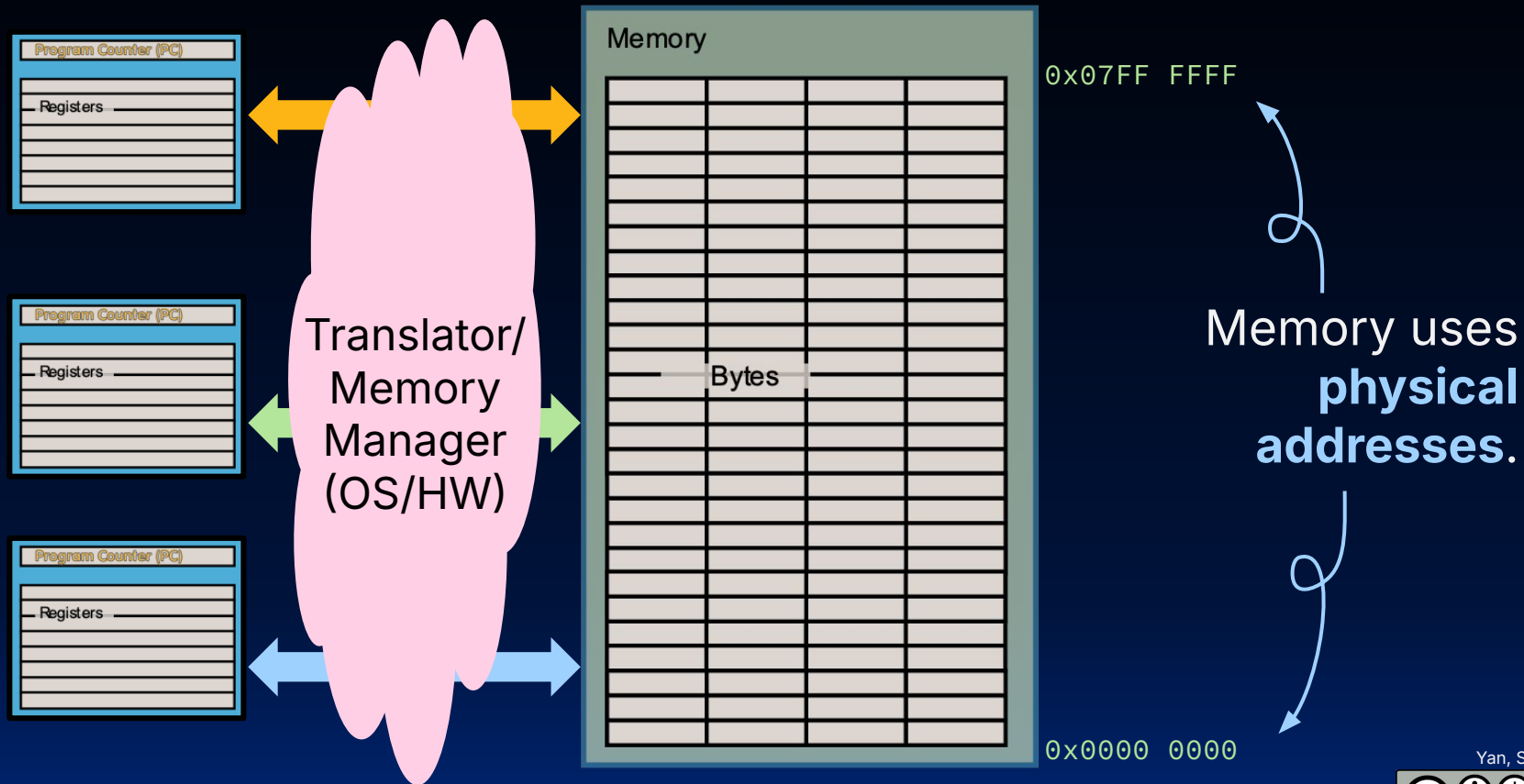
Different processes run simultaneously



Processes use **virtual addresses**.
Many processes, all using same (conflicting) addresses

The Illusion of Virtual Address Space (2/2)

Different processes run simultaneously





Virtual Memory

- **Virtual memory** is the next level in the memory hierarchy:
 - Give each process the **illusion** of a **full memory address space** that it has completely for itself.
 - Under the hood: working set of **pages** reside in main memory; other pages are on **disk**.
- Benefits:
 - Hides differences between machine configurations.
 - **Demand paging** provides the ability to run programs larger than the primary memory (DRAM). [Case 1]
 - OS can share memory and **protect** programs from each other. [Case 2; most important today]

Virtual Addresses vs. Physical Addresses

- **Virtual Address Space**

- Set of addresses that the user program knows about

- **Physical Address Space**

- Set of addresses that map to actual physical locations in memory
- Hidden from user applications



of yore. And the creation of a new type of administrator measured both by the money he or she is able to attract and by the degree of his or her benign neglect of educational policy issues. The president teaching moral philosophy gave way to the president with hat in hand.

(Approximate Analogy:
PDF Reader with many
different chapters)

KERR, C. (2001). The Uses of the University. Harvard University Press.
<http://www.jstor.org/stable/j.ctt6wpqkr>

Yan, SP26



Paged Memory

- MIMD: implementation —
 - Cache Coherence
 - Synchronization with Locks
- Virtual Memory —
 - Virtual Memory and Virtual Addresses
 - Paged Memory

OS Responsibilities: Virtual Memory Management

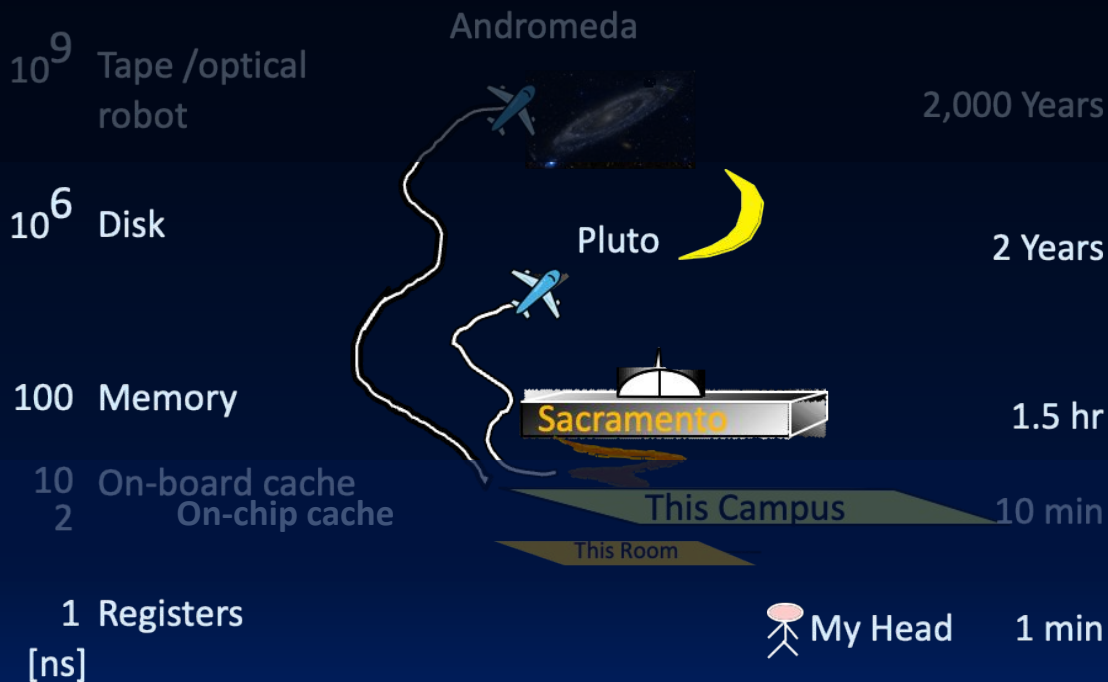
1. Address translator: map virtual addresses to physical addresses.
2. Use **both memory and disk**.
 - Give illusion of larger memory by storing some content on disk.
 - Disk is usually much larger and slower than DRAM.
3. Protection:
 - Isolate memory between processes.



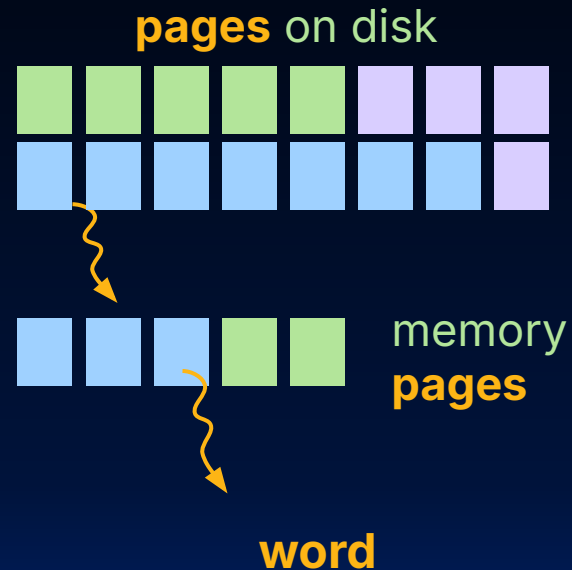
Recall principles of spatial
and temporal locality!
Fetch **pages** from disk.

Demand Paging

How Far Away is the Data?



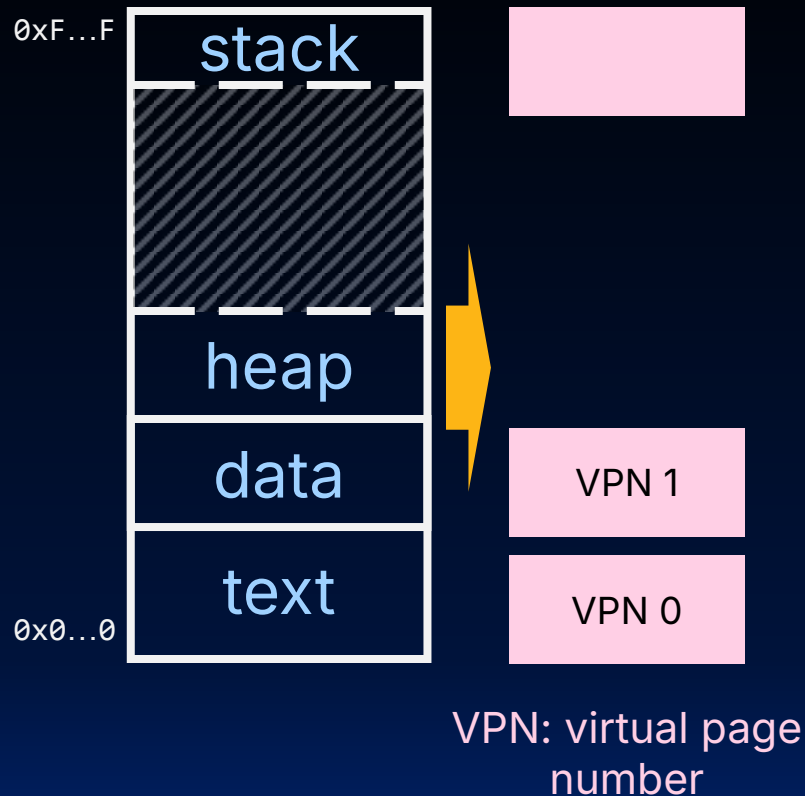
Spatial/temporal locality:
Store likely-to-access data on memory. Fetch from disk sparingly.



Paged Memory (1/2)

- The concept of “**paged memory**” dominates:
 - A disk access loads an entire page into memory (DRAM).
- Typical page size: **4-16 KiB**
- To address 4KiB bytes on page, need 12 bits of **page offset**

```
$ cat /proc/cpuinfo
$ getconf PAGESIZE # 4096
```





Paged Memory (2/2)

- The concept of “**paged memory**” dominates:
 - A disk access loads an entire page into memory (DRAM).
- Typical page size: **4-16 KiB**
- To address 4KiB bytes on page, need 12 bits of **page offset**

- Memory translation maps a **Virtual Page Number (VPN)** to a **Physical Page Number (PPN)**.

Virtual address (e.g. 64 bits)

VPN (52 bits)

offset (12 bits)

Physical address (e.g. 34 bits)

PPN (22 bits)

offset (12 bits)

Virtual and physical pages are **the same size**. Same # bits to address offset within a page!

```
$ cat /proc/cpuinfo
$ getconf PAGESIZE # 4096
```

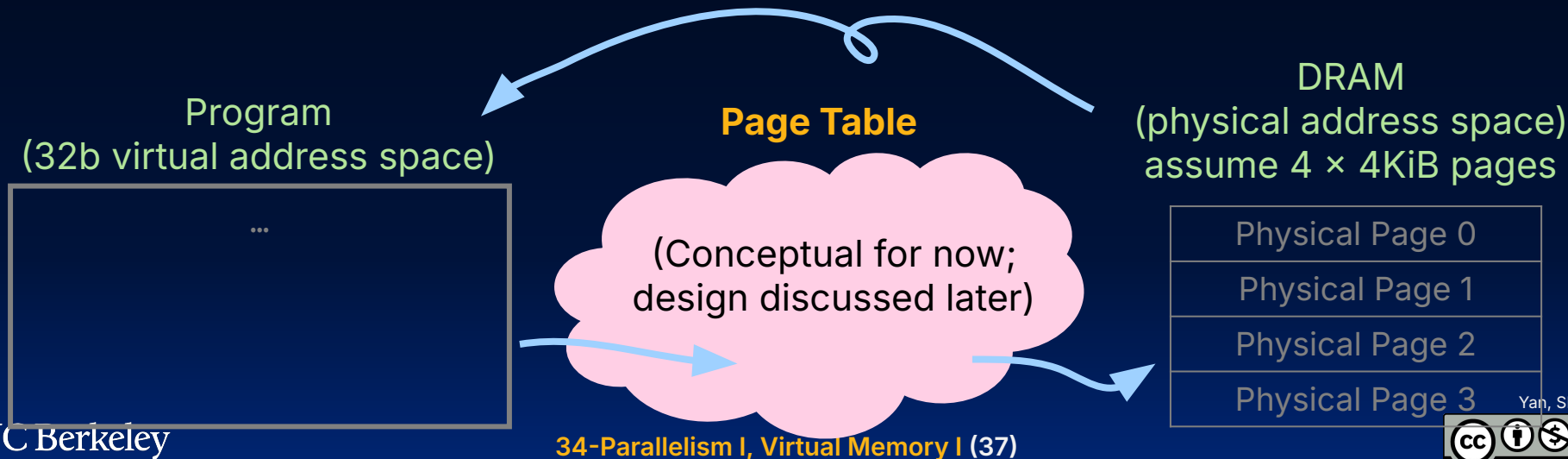


Address Translation

- MIMD: implementation —
 - Cache Coherence
 - Synchronization with Locks
- Virtual Memory —
 - Virtual Memory and Virtual Addresses
 - Paged Memory

Address Translation

- A **page table** translates virtual addresses to physical addresses for a given process. Each entry in the table:
 - Corresponds to a virtual page number.
 - If page is in memory, entry has corresponding physical page number.
 - Else, entry should tell OS to trigger **page fault** to **load page from disk**.



Address Translation I: Page Is in Memory (1/4)

1. Program executes a load specifying a **virtual address (VA)**.

Program
(32b virtual address space)

Load byte @ **0xFFFF F004**
to register t0

1

Page Table



DRAM
(physical address space)
assume $4 \times 4\text{KiB}$ pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is in Memory (2/4)

2

OS translates **VA** to the **physical address (PA)** in memory.

- Extract virtual page number (VPN) from VA
- Look up physical page number (PPN) in **page table**
- Construct PA: physical page number + offset (from virtual address)

(Conceptual image
for now; design
discussed later)

Program
(32b virtual address space)

Load byte @ 0xFFFF F004
to register t0

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

2

Address Translation I: Page Is in Memory (3/4)

3

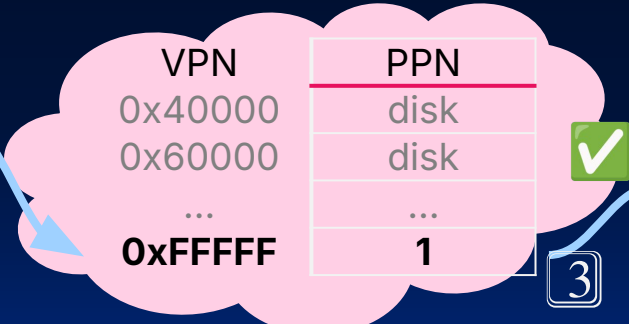
Page Fault: If ~~physical page~~ not in memory, then OS loads page from disk.

Page table entry is valid!
No page fault because
page is in memory.

Program
(32b virtual address space)

Load byte @ 0xFFFF F004
to register t0

Page Table



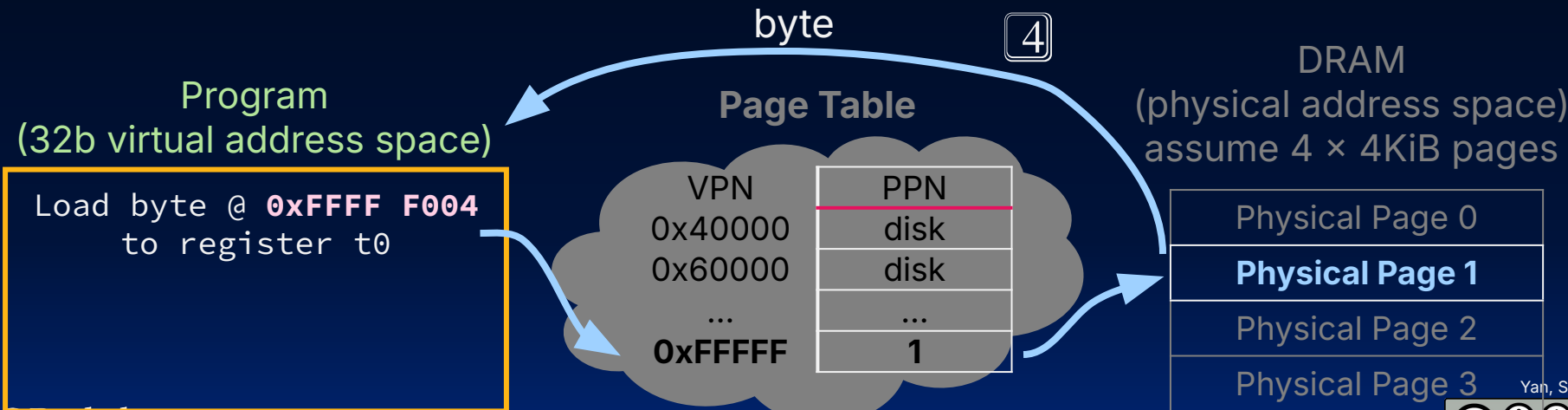
DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is in Memory (4/4)

- Read memory at the **physical address** and return the data to the program.



Address Translation I: Page Is Not in Memory (1/4)

1. Program executes a load specifying a **virtual address (VA)**.

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004
to register t0
Load byte @ 0x6000 0030
to register t0
```

1

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is Not in Memory (2/4)

2

OS translates **VA** to the **physical address (PA)** in memory.

- Extract virtual page number (VPN) from VA
- Look up physical page number (PPN) in **page table**
- Construct PA: physical page number + offset (from virtual address)

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004
to register t0
Load byte @ 0x6000 0030
to register t0
```

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

(still conceptual
image)

DRAM

(physical address space)
assume 4 × 4KiB pages

Physical Page 0

Physical Page 1

Physical Page 2

Physical Page 3

Yan, SP26

2

Address Translation I: Page Is Not in Memory (3/4)

3

Page Fault: If **physical page** not in memory, then OS loads page from disk.

Page fault! Need to load page from disk.

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004
to register t0
Load byte @ 0x6000 0030
to register t0
```

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1



3

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is Not in Memory (3/4)

3

Page Fault: If **physical page** not in memory, then OS loads page from disk.

Page fault! Need to load
page from disk.
(a million cycles later)

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004
to register t0
Load byte @ 0x6000 0030
to register t0
```

Page Table

VPN	PPN
0x40000	disk
0x60000	2
...	...
0xFFFFF	1



3

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is Not in Memory (4/4)

- Read memory at the **physical address** and return the data to the program.

